



TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 3 – Software Requirement Specification

– Danang, September 2025 –

Table of Contents

I. Record of Changes	3
II. Software Requirement Specification	4
1. Product Overview	4
2. User Requirements	5
2.1 Actors	5
2.2 Use Cases	6
3. Functional Requirements	12
3.1 System Functional Overview	12
3.2 Customer Module	15
3.3 Staff Module	20
3.4 Kitchen Staff Module	25
3.5 Admin Module	28
3.6 AI Module	30
4. Non-Functional Requirements	32
4.1 External Interfaces	32
4.2 Quality Attributes	33
5. Requirement Appendix	35
5.1 Technologies	35
5.2 Installation & Setup Guide	36
5.3 Contributors	37

I. Record of Changes

Date	A/M/D	In charge	Change Description
15/09/2025	A	Truong Quang Huy	Initial SRS Document for RMS
18/09/2025	M	Nguyen Duc Thuong	Update Architecture & Technologies
20/09/2025	A	Pham Chau Vinh	Add Order Processing APIs details
22/09/2025	A	Pham Van Quyet	Add Frontend UI/UX details
25/09/2025	A	Phan Nguyen	Add Payment Integration details

Table 1 - Record of Changes

II. Software Requirement Specification

1. Product Overview

The Restaurant Management System (RMS) is a comprehensive restaurant management platform that allows customers to order food online, reserve tables, make online payments, and review restaurant services. Traditional restaurant operations often face challenges with manual order taking, inefficient table management, and slow communication between front-of-house and kitchen staff.

This system helps restaurant staff manage tables, process customer orders, generate invoices, and support customers in real time. Kitchen staff can monitor cooking orders and update food preparation status. Administrators can manage menus, employees, coupons, revenue statistics, and customer feedback.

The system is designed to improve restaurant operation efficiency, enhance customer experience, and support digital transformation in restaurant management.

The context diagram below illustrates the system boundaries of the RMS and its interactions with external entities, including Customers, Staff, Kitchen Staff, and Admin, as well as third-party services such as Payment Gateways, AI APIs, and Cloud Storage.

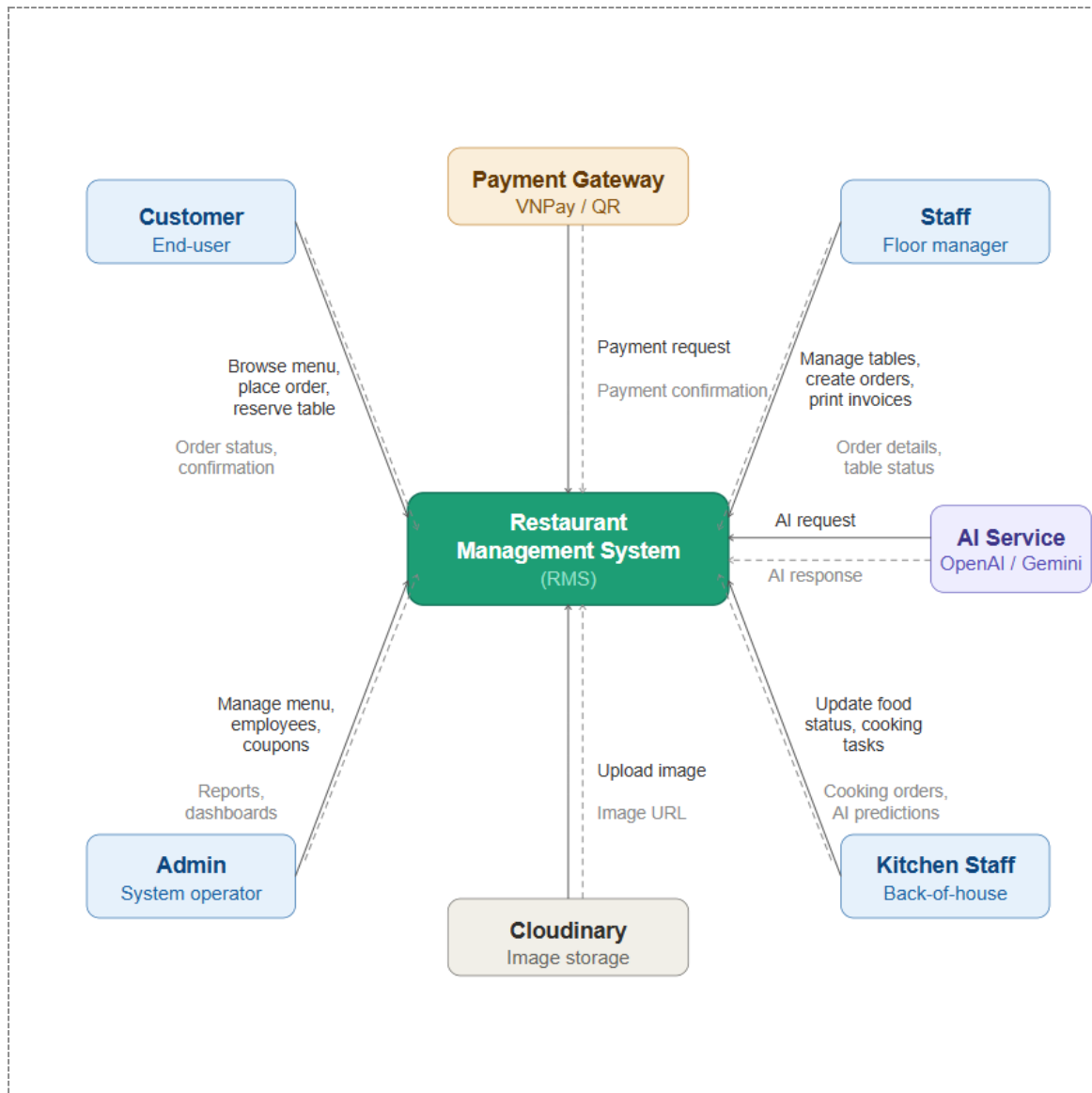


Figure 1 - Context Diagram

2. User Requirements

2.1 Actors

#	Actor	Description
1	Customer	A Customer is an end-user of the system who can view menus, search food, filter by category, place orders, apply coupons, reserve tables, make online payments, review food, and chat with the AI chatbot for recommendations.
2	Staff	A Staff member is responsible for managing the dining area. They can manage tables, create orders for walk-in customers, update order statuses, transfer tables, confirm payments, print invoices, and handle customer requests in real time.
3	Kitchen Staff	A Kitchen Staff member operates in the back-of-house. They can view incoming cooking orders, accept or cancel cooking tasks, update food preparation status, and use AI to predict cooking time and sequence.
4	Admin	An Admin is the system operator responsible for overall management. They can manage employees, roles, food menus, image uploads, coupons, view revenue statistics and dashboards, and monitor customer feedback.

Table 2 - Actors Description

2.2 Use Cases

2.2.1 Diagram(s)

The following use case diagram illustrates the interactions between the four main actors (Customer, Staff, Kitchen Staff, Admin) and the system's key functionalities.

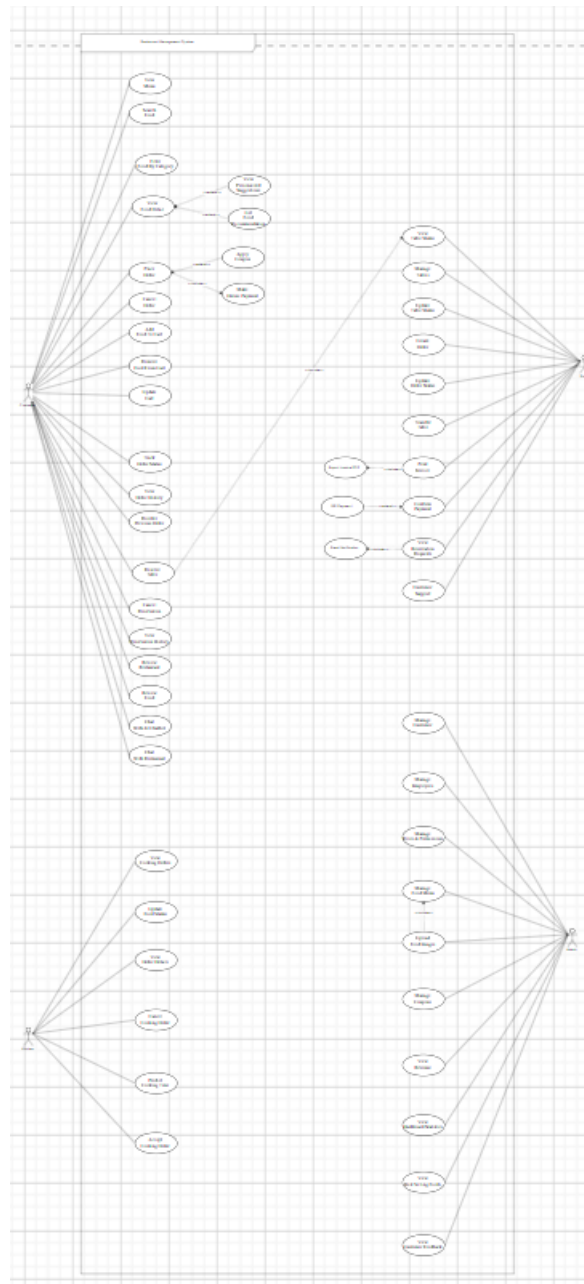


Figure 2 - Use Cases Overview

2.2.2 Descriptions

ID	Use Case	Actors	Use Case Description
01	View Menu	Customer	Customer views the available food menu of the restaurant.
02	Search Food	Customer	Customer searches for specific food items using keywords.
03	Filter Food By Category	Customer	Customer filters the food items based on categories (e.g. drinks, main course).

04	View Food Detail	Customer	Customer views the details, ingredients, and price of a specific food item.
05	Add Food To Cart	Customer	Customer adds selected food items to their shopping cart.
08	Apply Coupon	Customer	Customer applies a discount code during checkout.
09	Place Order	Customer	Customer places a final order for the items in their cart.
11	Track Order Status	Customer	Customer tracks the real-time status of their placed order.
14	Make Online Payment	Customer	Customer pays for their order securely via online payment gateways.
15	Reserve Table	Customer	Customer books a table for a specific date, time, and number of guests.
18	Review Food	Customer	Customer submits a review and rating for the food they ordered.
21	Chat With AI Chatbot	Customer	Customer interacts with an AI chatbot for quick support and recommendations.
25	Manage Tables	Staff	Staff updates table availability and assigns customers to tables.
27	Create Order	Staff	Staff creates an order manually for dine-in customers.
28	Update Order Status	Staff	Staff updates the status of an order (e.g. served, pending payment).
34	View Cooking Orders	Kitchen Staff	Kitchen staff views the queue of orders that need to be prepared.
35	Update Food Status	Kitchen Staff	Kitchen staff marks food items as preparing, ready, or delayed.
38	Predict Cooking Time	Kitchen Staff	System AI assists kitchen staff in predicting the estimated cooking time.
41	Manage Employees	Admin	Admin adds, updates, or removes staff and kitchen accounts.
43	Manage Food Menu	Admin	Admin updates the restaurant menu, pricing, and availability.
46	View Revenue	Admin	Admin views revenue reports and financial statistics.

Table 3 - Use Case Description

3. Functional Requirements

3.1 System Functional Overview

The RMS platform is built with a micro-services oriented architecture to ensure separate functionality for the four main user roles. Features like QR Menu, QR Payment, and automated Email Notifications are integrated to provide a seamless flow. AI algorithms are deeply integrated to provide smart recommendations, time predictions, and analytics. The system architecture diagram below illustrates the overall structure and interaction between the frontend, backend services, database, and third-party integrations.

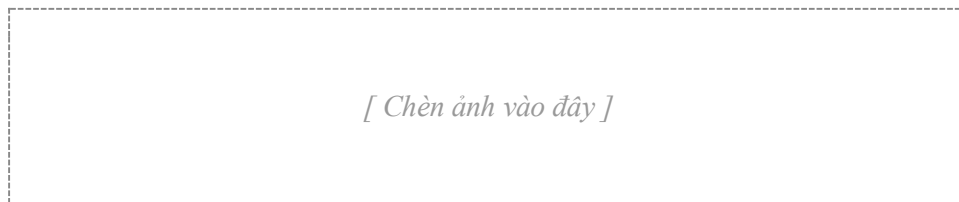


Figure 3 - System Architecture Overview

3.2 Customer Module

The Customer module provides all the tools necessary for an end-user to interact with the restaurant digitally:

- Xem menu món ăn, Tìm kiếm và lọc món ăn, Xem chi tiết món ăn
- Thêm món vào giỏ hàng, Áp dụng mã giảm giá
- Đặt món online, Thanh toán online
- Theo dõi trạng thái đơn hàng, Xem lịch sử đơn hàng
- Đặt bàn online, Hủy đặt bàn
- Đánh giá món ăn và nhà hàng
- Chat với nhà hàng, Chat với AI chatbot
- Nhận gợi ý món ăn bằng AI, Sử dụng QR menu và QR thanh toán

The interface mockup below demonstrates the Customer-facing screens including the food menu, cart, order tracking, and table reservation flows.



Figure 4 - Customer Module UI Mockup

3.3 Staff Module

The Staff module equips waiters and floor managers with real-time operational controls:

- Quản lý bàn ăn, Cập nhật trạng thái bàn, Tạo order cho khách tại quán, Cập nhật trạng thái order, Chuyển/gộp/tách bàn
- In hóa đơn, Xác nhận thanh toán
- Xử lý yêu cầu đặt bàn, Quản lý thông tin khách hàng, Hỗ trợ khách hàng realtime

- AI đề xuất bàn phù hợp, AI ưu tiên order cần xử lý



Figure 5 - Staff Module UI Mockup

3.4 Kitchen Staff Module

The Kitchen Staff module focuses on back-of-house efficiency and order fulfillment:

- Xem danh sách món cần chế biến, Theo dõi order realtime, Cập nhật trạng thái món ăn
- Quản lý quy trình chế biến
- AI sắp xếp thứ tự chế biến món ăn, AI dự đoán thời gian hoàn thành món, AI cảnh báo nguyên liệu sắp hết



Figure 6 - Kitchen Staff Module UI Mockup

3.5 Admin Module

The Admin module provides high-level control and analytical insights for restaurant owners:

- Quản lý người dùng, Quản lý nhân viên, Phân quyền hệ thống
- Quản lý menu món ăn, Quản lý danh mục món ăn, Upload hình ảnh món ăn
- Quản lý mã giảm giá, Quản lý doanh thu, Xem dashboard thống kê, Xem báo cáo đơn hàng
- Quản lý đánh giá khách hàng, Theo dõi món ăn bán chạy
- AI phân tích đánh giá khách hàng, AI dự đoán doanh thu, AI phân tích hành vi khách hàng, AI tự động phản hồi review



Figure 7 - Admin Module Dashboard Mockup

3.6 AI Module

A dedicated AI subsystem enhances both customer experience and operational efficiency:

- Hỗ trợ chatbot tự động, Gợi ý món ăn phù hợp, Gợi ý combo món ăn, Tìm kiếm món ăn thông minh
- Phân tích đánh giá khách hàng, Dự đoán doanh thu, Phân tích hành vi khách hàng, Tối ưu xử lý order và chế biến món ăn

[Chèn ảnh vào đây]

Figure 8 - AI Module Flow Diagram

4. Non-Functional Requirements

4.1 External Interfaces

- Swagger UI: API Documentation accessed via <http://localhost:8080/swagger-ui.html> for Authentication, Food, Order, Reservation, Payment, and Customer APIs.
- Cloudinary: Integrated for seamless and optimized image uploads (food images, user avatars).
- Payment Gateways: Integration for online payment processing (QR Payment).
- OpenAI / Gemini API: External AI services used to power the chatbot and recommendation algorithms.

4.2 Quality Attributes

- Role-Based Access Control (RBAC): Strict security implementation using Spring Security + JWT to ensure Admin, Staff, Kitchen Staff, and Customers only access their respective endpoints.
- Real-time Capabilities: Real-time order tracking and chat functionality for instant communication.
- Automated Notifications: High reliability in dispatching automated email or in-app notifications for order statuses and payments.
- Responsive UI: The frontend built with ReactJS and Bootstrap 5 must be fully responsive across desktop, tablet, and mobile browsers.

5. Requirement Appendix

5.1 Technologies

Layer	Technology Stack
Backend	Java 21, Spring Boot, Spring Security + JWT, Spring Data JPA / Hibernate, Maven
Frontend	ReactJS, Bootstrap 5, Axios
Database	MySQL 8+
AI Integration	OpenAI API / Gemini API
Other Tools	Swagger UI, Cloudinary, GitHub, Postman, Figma

Table 4 - Technologies Stack

5.2 Installation & Setup Guide

Prerequisites:

- Java 21
- Node.js
- MySQL 8+
- Maven 3.9+
- Git
- Optional: Cloudinary Account (for image upload feature)

Setup Steps:

Step 1: Clone Repository

```
git clone https://github.com/your-repo/restaurant-management-system.git
```

Step 2: Database Setup

```
CREATE DATABASE restaurant_management_db;
```

Step 3: Configure Environment

Update src/main/resources/application.properties with database and mail credentials.

Step 4: Run Backend

```
mvn clean install
```

```
mvn spring-boot:run (Runs at localhost:8080)
```

Step 5: Run Frontend

```
npm install
```

```
npm run dev (Runs at localhost:5173)
```

5.3 Contributors

Member	Student ID	Responsibilities
Nguyễn Đức Thương	DE190096	Backend Development, Database Design, Authentication APIs
Phạm Châu Vinh	DE190156	Order Processing APIs, Backend Development
Phạm Văn Quyết	DE190425	Frontend Development, UI/UX Design
Phan Nguyễn	DE191019	Payment Integration, Customer APIs

Truong Quang Huy	DE190381	SRS Documentation, Testing, Project Management
------------------	----------	--

Table 5 - Contributors